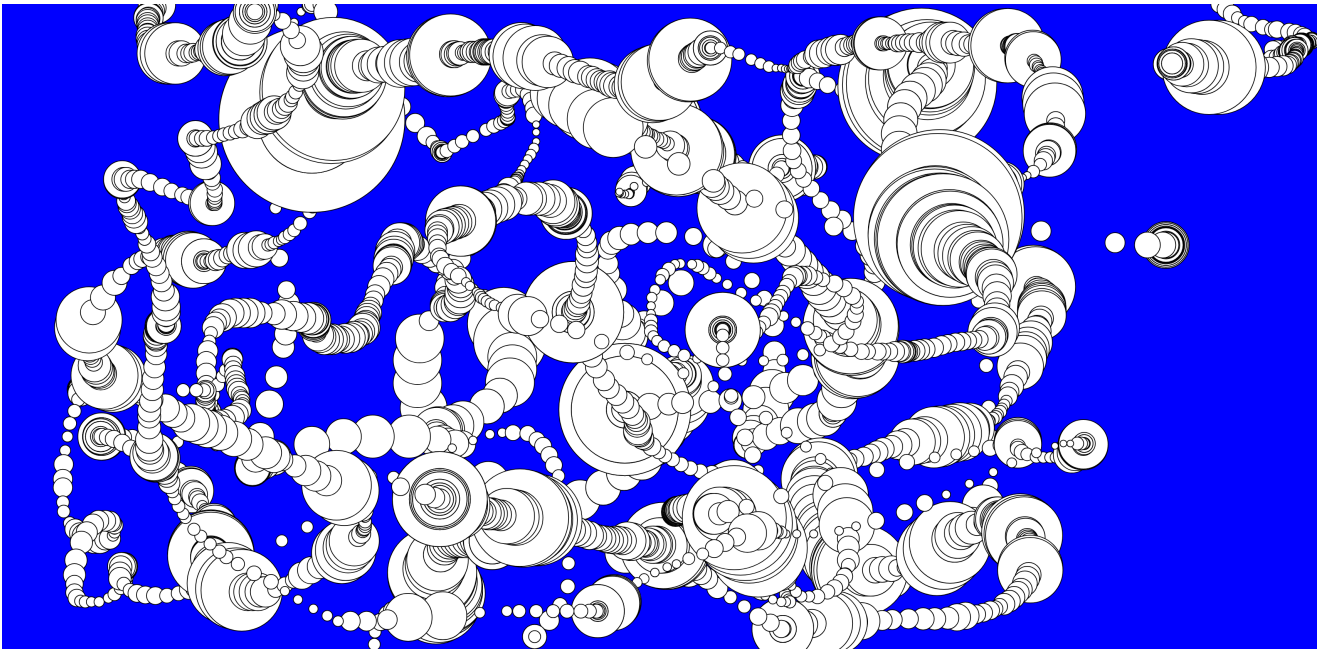


Audioreactive Pen (with Ted)



```
// {"P5LIVE":{"name":"01_audio reactive pen","mod":1777906728974}}

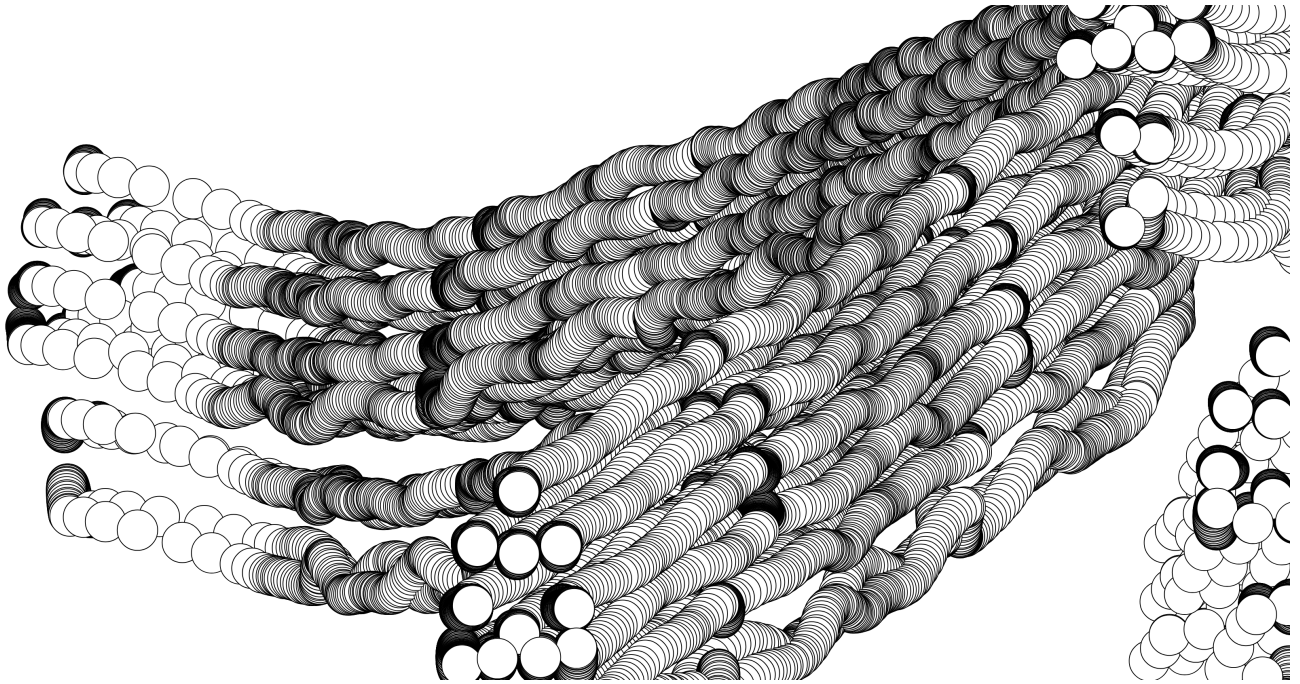
// control shift A = audioreactive snippet

function setup() {
  createCanvas(windowWidth, windowHeight)
  background(0, 0, 255)
  setupAudio(true) // global vars
  // a5.ease = .075 // set easing
}

function draw() {
  /* audio vars: amp, ampEase, fft, fftEase, waveform, waveformEase */
  updateAudio()
  circle(mouseX, mouseY, amp)
}

}
```

Library Download – Walker (with Ted)



```
// {"P5LIVE":{"name":"08_Walker","mod":1777911913129}}

let libs =
['https://tetunori.github.io/BMWalker.js/dist/v0.6.1/bmwalker.js']
  const bmw = new BMWalker();

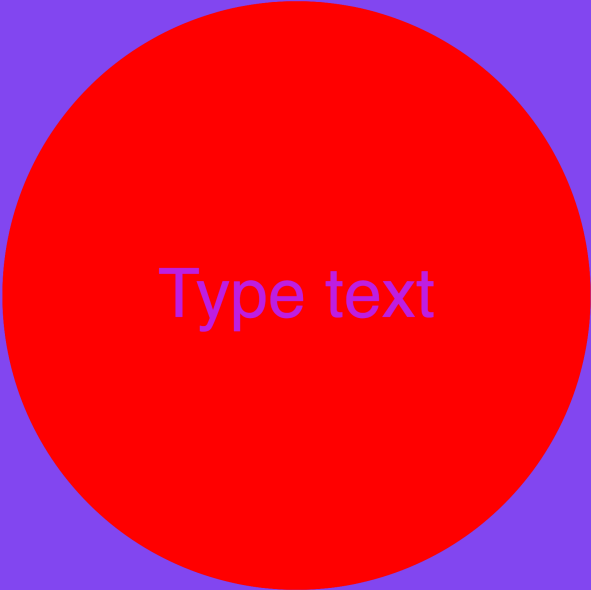
function setup() {
  createCanvas(windowWidth, windowHeight)

}

function draw() {
  //clear()
  const walkerHeight = 500;
  const markers = bmw.getMarkers(walkerHeight);

  //translate(width/2, height/2)
  translate(mouseX, mouseY)
  markers.forEach((m) => {
    circle(m.x, m.y, 50);
  });
}
```

Buttons, Checkboxes, Sliders, Dropdown, Randomizers (with Alper)



Type text

```
// {"P5LIVE":{"name":"09_AlperMenues","mod":1777988226391}}

let slider, checkbox, button, colorPicker, dropdown, input, sliderText,
buttonText, radioButton;
let bgColor;
let colorText;
let positionDOM;

function setup() {
  createCanvas(windowWidth, windowHeight);
  positionDOM = width - 400

  //checkbox
  checkbox = createCheckbox('Show Form', true);
  checkbox.position(width - 400, 20);

  //sliders
  slider = createSlider(100, height - 100, 200);
  slider.position(positionDOM, 60);

  sliderText = createSlider(20, 300, 200);
  sliderText.position(positionDOM, 260);

  // Button
  button = createButton('Random Background');
  button.position(positionDOM, 100);
  button.mousePressed(() => {
    bgColor = color(random(255), random(255), random(255));
  });
  bgColor = color(220);
```

```

// Button Text
buttonText = createButton('Random Fontcolor');
buttonText.position(positionDOM, 300);
buttonText.mousePressed(() => {
    colorText = color(random(255), random(255), random(255));
});
colorText = color(220);

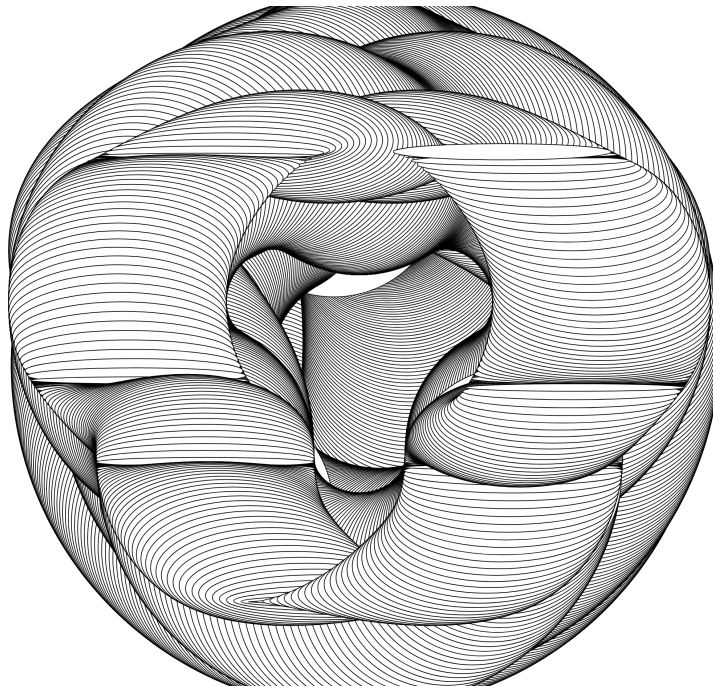
// Color Picker
colorPicker = createColorPicker('#ff0000');
colorPicker.position(positionDOM, 140);

// Dropdown
dropdown = createSelect();
dropdown.position(positionDOM, 180);
dropdown.option('Circle');
dropdown.option('Square');
rectMode(CENTER);

// Input field
input = createInput('Type text');
input.position(positionDOM, 220);
textAlign(CENTER, CENTER);
}

```

Sin/Cos Explanation (with Alper)



```

// {"P5LIVE":{"name":"new_001","mod":1777989596135}}

function setup() {
    createCanvas(windowWidth, windowHeight)
}

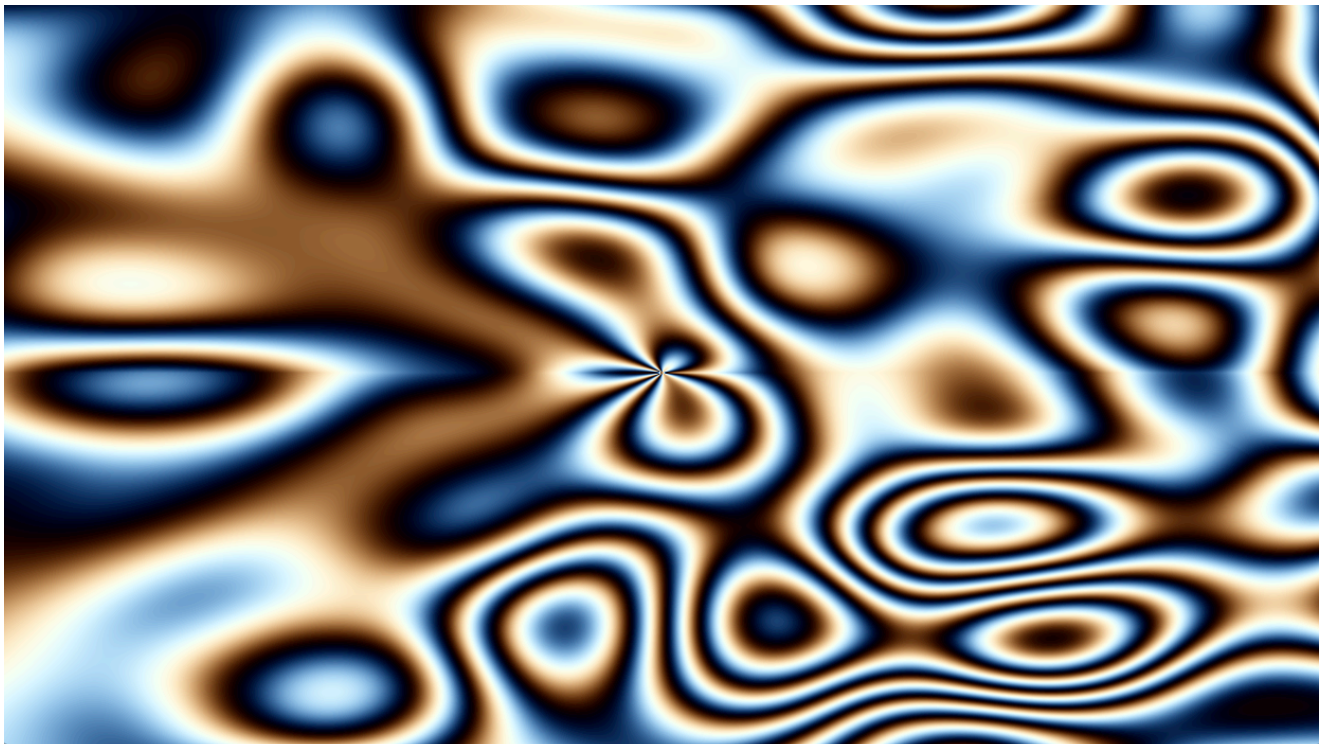
```



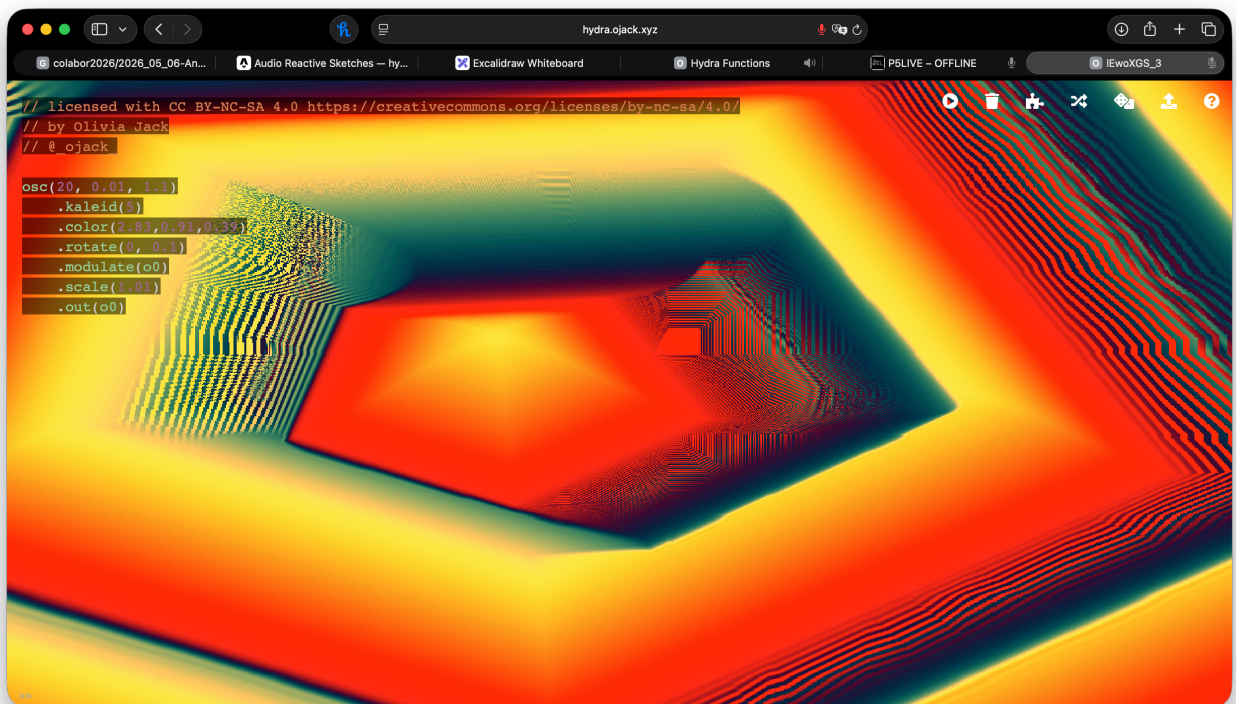
```
}
```

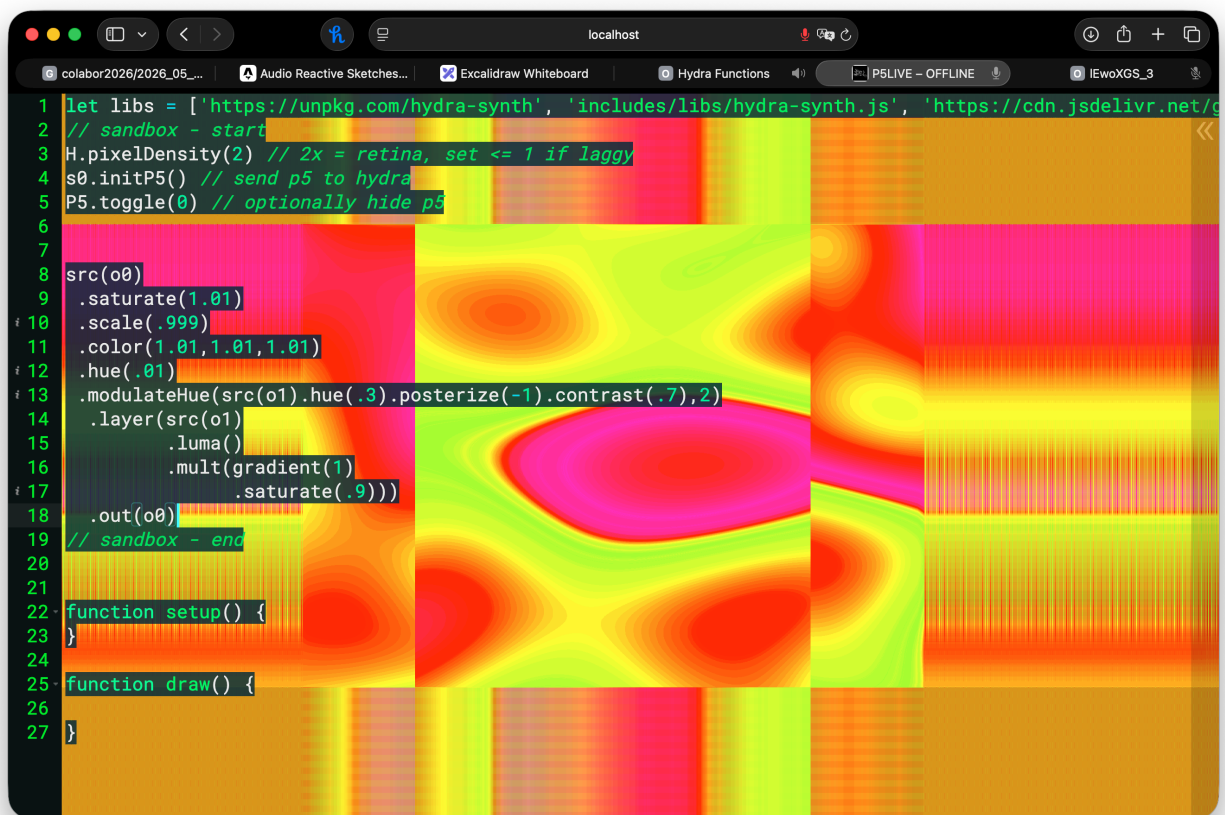
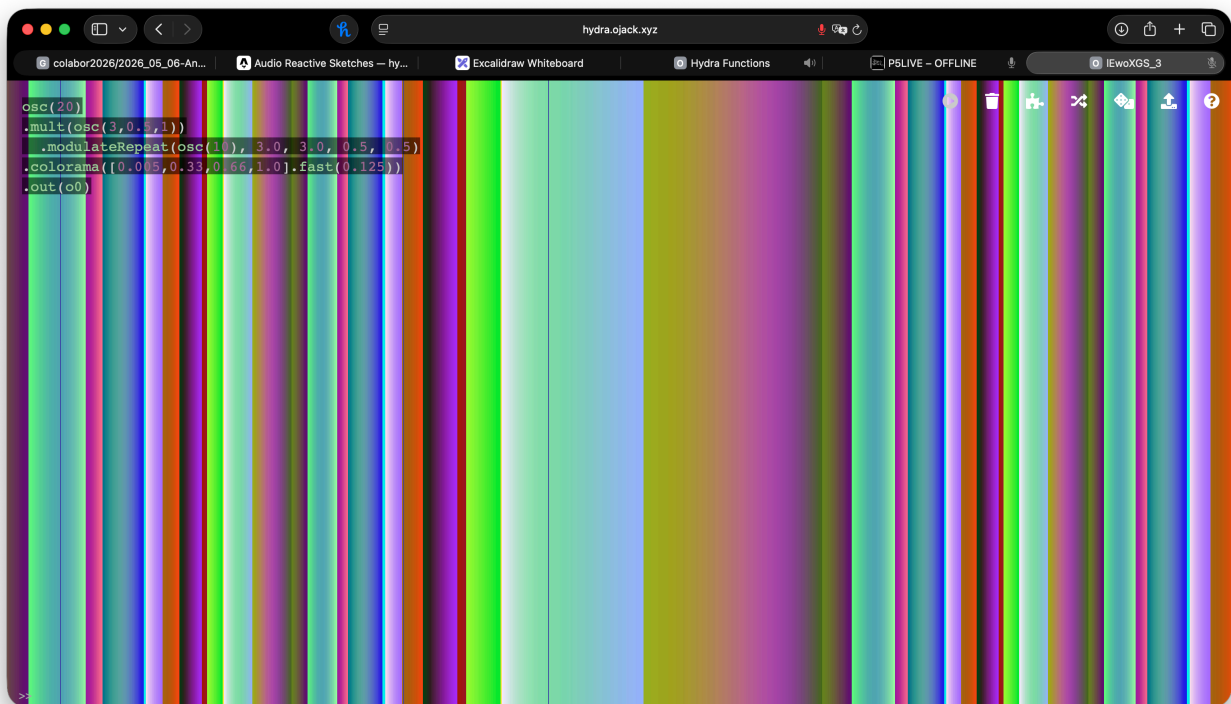
```
function draw() {  
  let speedX = sin(frameCount * 0.02) * 300 + sin(frameCount * 0.08) * 50  
  
  let speedY = cos(frameCount * 0.02) * 300 + cos(frameCount * 0.08) * 50  
  
  //ellipse(width/2+speedX, height/2+speedY,300, sin(frameCount * 0.08) *150  
  )  
}
```

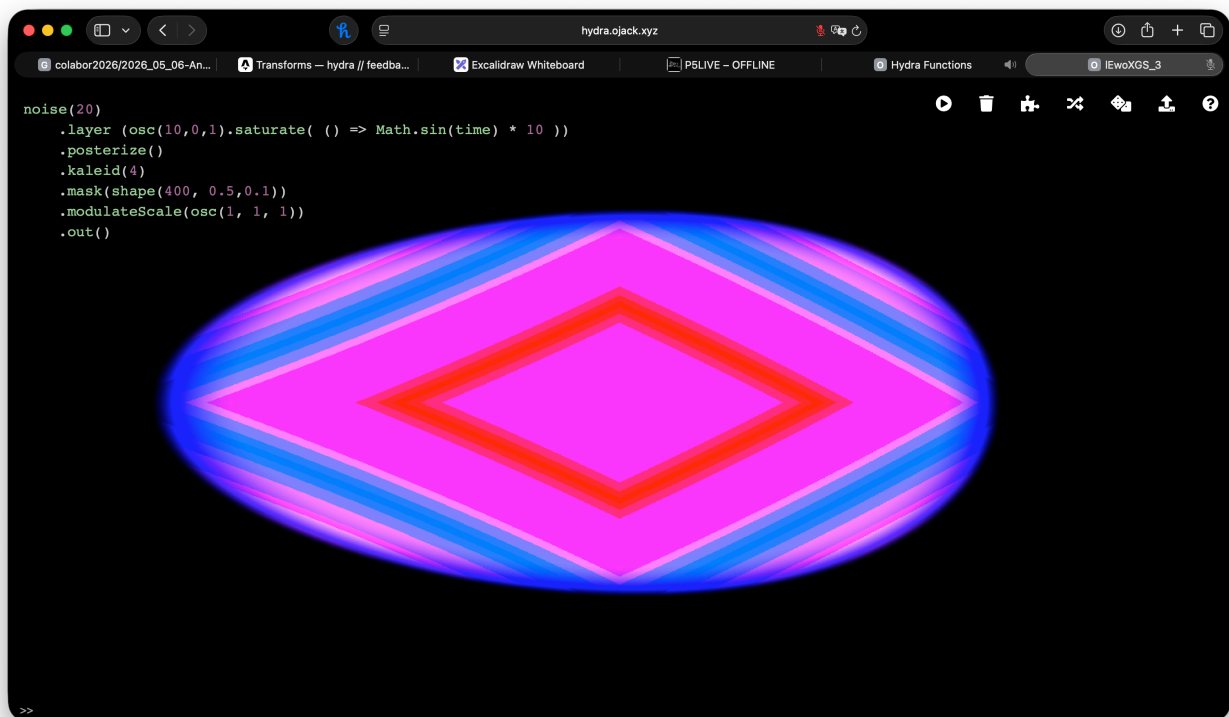
Hydra/Hy5 (with Andrea)



```
1 osc(20,1,.4,.2).modulateKaleid(noise(3), 0.3).out()
```

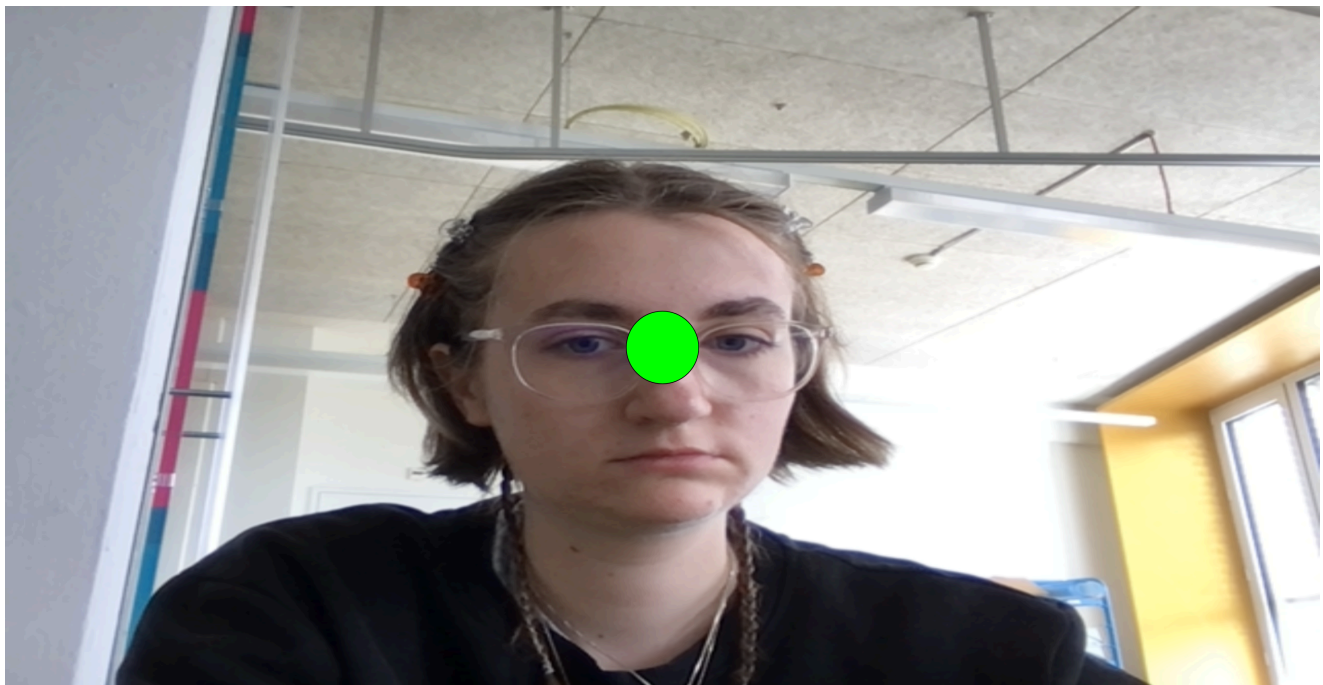






Machine Learning (with Paulina)

Bottle, Phone, Me => Recognition



```
<div>Teachable Machine Image Model</div>
<button type="button" onclick="init()">Start</button>
<div id="webcam-container"></div>
<div id="label-container"></div>
<script
src="https://cdn.jsdelivr.net/npm/@tensorflow/tfjs@latest/dist/tf.min.js">
</script>
<script
```



```

src="https://cdn.jsdelivr.net/npm/@teachablemachine/image@latest/dist/teac
hablemachine-image.min.js"></script>
<script type="text/javascript">
  // More API functions here:
  // https://github.com/googlecreativelab/teachablemachine-
community/tree/master/libraries/image

  // the link to your model provided by Teachable Machine export panel
  const URL = "https://teachablemachine.withgoogle.com/models/-
rxWo3h/";

  let model, webcam, labelContainer, maxPredictions;

  // Load the image model and setup the webcam
  async function init() {
    const modelURL = URL + "model.json";
    const metadataURL = URL + "metadata.json";

    // load the model and metadata
    // Refer to tmImage.loadFromFiles() in the API to support files
from a file picker
    // or files from your local hard drive
    // Note: the pose library adds "tmImage" object to your window
(window.tmImage)
    model = await tmImage.load(modelURL, metadataURL);
    maxPredictions = model.getTotalClasses();

    // Convenience function to setup a webcam
    const flip = true; // whether to flip the webcam
    webcam = new tmImage.Webcam(200, 200, flip); // width, height,
flip

    await webcam.setup(); // request access to the webcam
    await webcam.play();
    window.requestAnimationFrame(loop);

    // append elements to the DOM
    document.getElementById("webcam-
container").appendChild(webcam.canvas);
    labelContainer = document.getElementById("label-container");
    for (let i = 0; i < maxPredictions; i++) { // and class labels
      labelContainer.appendChild(document.createElement("div"));
    }
  }

  async function loop() {
    webcam.update(); // update the webcam frame
    await predict();
    window.requestAnimationFrame(loop);
  }

```

```

// run the webcam image through the image model
async function predict() {
  // predict can take in an image, video or canvas html element
  const prediction = await model.predict(webcam.canvas);
  for (let i = 0; i < maxPredictions; i++) {
    const classPrediction =
      prediction[i].className + ": " +
prediction[i].probability.toFixed(2);
    labelContainer.childNodes[i].innerHTML = classPrediction;
  }
}
</script>

```

Text Patterns (with Jasmin)



Repetition & Replacement & Text Attributes

```

// {"P5LIVE":{"name":"12_textattributes","mod":1778161610117}}

function setup() {
  createCanvas(windowWidth, windowHeight)
}

function draw() {
  let live = frameCount % 10
  let sine = floor(5*sin(frameCount/10)+5)
  let words = ["xyx", "lol", "ihi", "ololo", "jaj", "ueu", "lsl",
"asa", "ere", "lil"]
  let rand = random(words);

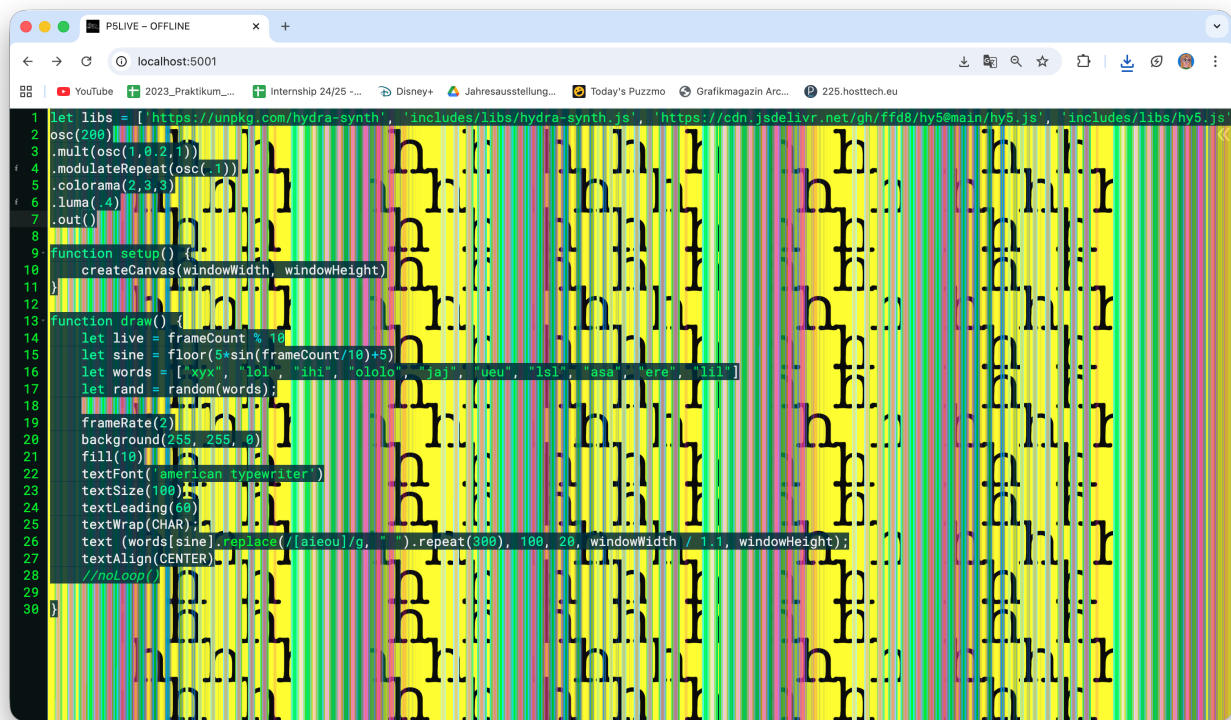
  frameRate(5)
}

```

```

background(255, 255, 0)
fill(10)
textFont('american typewriter')
textSize(100)
textLeading(60)
textWrap(CHAR);
text (words[sine].replace(/[aieou]/g, "w").repeat(200), 100, 20,
windowWidth / 1.1, windowHeight);
textAlign(CENTER)
}

```



Text switches, Hydra Overlay

```

let libs = ['https://unpkg.com/hydra-synth', 'includes/libs/hydra-synth.js', 'https://cdn.jsdelivr.net/gh/ffd8/hy5@main/hy5.js', 'includes/libs/hy5.js']
osc(200)
  .mult(osc(1,0.2,1))
  .modulateRepeat(osc(.1))
  .colorama(2,3,3)
  .luma(.3)
  .out()

function setup() {
  createCanvas(windowWidth, windowHeight)
}

function draw() {
  let live = frameCount % 10

```

```
let sine = floor(5*sin(frameCount/10)+5)
let words = ["xyx", "lol", "ihi", "ololo", "jaj", "ueu", "lsl",
"asa", "ere", "lil"]
let rand = random(words);

frameRate(2)
background(255, 255, 0)
fill(10)
textFont('american typewriter')
textSize(100)
textLeading(60)
textWrap(CHAR);
text (words[sine].replace(/[aieou]/g, " ").repeat(300), 100, 20,
windowWidth / 1.1, windowHeight);
textAlign(CENTER)

}
```